数学与 CS 问答笔记

answer-note

今天只保留一节：从零重建抽象函数、表示不变式与表示暴露，并讲它们在 Go 里的表达手段（ADT 形状、checkRep、存在性、所有权、错误、表驱动测试）；只讲知识与取舍，不包含项目答案或具体步骤。

October 24, 2026

Contents

1. 重讲与 Go 落地：从概念到语言手段（当前步骤对齐） 3
   1.1. 〇、本轮边界 3
   1.2. 一、重建概念：先回答三个问题 3
   1.3. 二、Go 手段（一）：ADT 长什么样 4
   1.4. 三、Go 手段（二）：AF、RI 与检查函数写在哪 4
   1.5. 四、Go 手段（三）：表达“存在性” 5
      1.5.1. 语言事实（可查证） 5
   1.6. 五、Go 手段（四）：表达“所有权” 5
      1.6.1. 一个容易忽略的对称性 6
   1.7. 六、Go 手段（五）：表达“错误”与“不存在” 6
   1.8. 七、Go 手段（六）：把测试分区写出来 7
   1.9. 八、当前步骤的“语言工具箱” 8
   1.10. 九、常见错误（概念类 / Go 类 / 学习类） 8
   1.11. 十、关键点与自检 8
   1.12. 十一、概念变式（不指向实现） 9
   1.13. 十二、同类归类 9
   1.14. 十三、Go 机制补充：查证清单 9
   1.15. 十四、本次操作 10
2. map 类表示的抽象函数怎么写（AF for Map-shaped Reps） 11
   2.1. 费曼直觉 11
   2.2. 硬核机制 11
      2.2.1. 一、AF 的左边与右边 11
      2.2.2. 二、map 类 AF 的三种标准写法 11
      2.2.3. 三、空 map、nil map 与定义域 12
      2.2.4. 四、当抽象值不是“映射”时 12
      2.2.5. 五、常见错误 12
      2.2.6. 六、自检三问 12
   2.3. 当前步骤：`mem` 的三个待答问题 13
   2.4. 关键点 13
   2.5. 变式检验 13
   2.6. 同类归类 13
   2.7. Go 语言机制补充 13
   2.8. 本次操作与当前状态 14

Index of Listings 15

1. 重讲与 Go 落地：从概念到语言手段（当前步骤对齐）

1.1. 〇、本轮边界

你说概念有遗忘，这一节从零重建三个概念，然后讲它们在 Go 里**有哪些表达手段**、各自的语义与代价。仍然遵守教练契约：**讲手段与取舍，不替你选，不贴 `kv` 的函数体**。如果你想直接要实现，明确说“解除契约”即可。

本文件只保留这一节，不包含任何练习的参考写法；写完后再自己复盘。

1.2. 一、重建概念：先回答三个问题

看任何一个抽象数据类型（abstract data type, ADT），先问三句话：

1. **它对外承诺什么？**——客户端能观察到的值，叫**抽象值（abstract value）**，记作集合 A。
2. **它内部用什么数据实现？**——字段的真实取值，叫**表示值（representation value）**，记作集合 R。
3. **两者怎么对应？哪些内部状态合法？客户端能不能绕过方法改内部？**

第三问拆成三件互不替代的事：

1. **抽象函数（abstraction function, AF）**：把 rep 值翻译成抽象值，$\mathrm{AF} : R \to A$；
2. **表示不变式（rep invariant, RI）**：判断 rep 值是否合法，$\mathrm{RI} : R \to \mathrm{boolean}$；
3. **表示暴露（rep exposure）**：客户端能不能拿到并修改 rep 的可变部分。

用“翻译官 / 门卫 / 围墙”记：

1. AF 是翻译官：把内部数据翻译成客户端听得懂的话；
2. RI 是门卫：不合法的内部状态根本不许进来；
3. 表示暴露是围墙缺口：客户端绕过正门，直接伸手改内部。

三者有一条硬关系：

> 📌 **核心要点 RI(r) 为真，当且仅当 AF(r) 有定义。** AF 常常是**偏函数（partial function）**——不是所有 rep 值都有意义；RI 就是把这些“没有意义”的 rep 值挡在门外的那道谓词。AF 还是**满射（surjective）**但不必**单射（injective）**：每个抽象值至少有一种表示，同一种抽象值也可以有多种表示。

课程里的 CharSet 是最好的例子：rep 是一个字符串，抽象值是字符集合。同一个字符串 rep，可以规定“不许重复字符”（一种 RI），也可以规定“字符非递减”（另一种 RI），还可以把“每两个字符”解释成一个区间（另一种 AF）。**类型只决定 R，不决定 RI 与 AF**——后两者是设计决策。

再记一个判据：**AF 不是“这段代码大概在干什么”的说明，而是一个可以代入的函数。**代入一个合法 rep 值，右边必须算出唯一一个抽象值；如果代进去右边一字不变，它就不是函数。

1.3. 二、Go 手段（一）：ADT 长什么样

Go 没有类。ADT 的惯用形状是：**结构体 + 非导出字段 + 方法**，封装边界是**包（package）**，不是类。

```go
package demo

// Counter 是一个只增不减的计数器。
type Counter struct {
  n int // 非导出字段：包外看不见
}

func (c *Counter) Add(d int) { c.n += d }

func (c *Counter) Value() int { return c.n }
```

Listing 1: Go 里 ADT 的最小形状：非导出字段 + 方法；没有类，封装靠包边界与字段大小写

三层理解：

1. **为什么这样设计**：Go 把“谁能看见”落在标识符首字母大小写上，而不是 `private` 关键字；包是最小封装单位。
2. **语言/运行时怎么实现**：结构体是连续内存布局，方法只是带接收者的函数；非导出字段在包外不可见是编译期规则，不是运行时检查。
3. **日常怎么写**：字段小写、类型导出；构造函数用 `NewXxx` 或项目里的 `Open`；返回值优先 `(T, error)`；零值尽量可用。

与课程材料的映射：材料的 `class` → Go 的结构体；`private` → 小写字段；`constructor` → `NewXxx` / `Open`；`this` → 方法接收者。材料里的继承在 Go 里没有对应物，用组合或嵌入替代。

1.4. 三、Go 手段（二）：AF、RI 与检查函数写在哪

三个位置规则：

1. AF/RI/安全性论证写在**字段声明旁边**，用普通注释；
2. **不要**写成 `go doc` 能看到的文档注释——那会把内部表示变成公开承诺；
3. 检查函数写成**非导出方法**（如 `checkRep`），不要导出。

为什么？`go doc` 展示的是**规格**，规格只能谈客户端可见的东西；AF/RI 是实现的内部约定，写进公开文档等于承诺“表示永远长这样”，以后换 rep 就被自己的文档绑住。

Go 没有 `assert`（语言事实，不是能推出来的约定）。两种替代：

1. 生产代码：`if !cond { panic(...) }`，因为 RI 被破坏是**程序员 bug**，不是调用方输入错误；
2. 测试代码：`if got != want { t.Fatalf(...) }`，或 testify。

检查函数的调用时机：**创建者、生产者、变值器**在返回前调用；**观察者**也建议调用，因为能抓住“客户端通过表示暴露偷偷改坏了 rep”的情况。

```
1    // Abstraction function:
2    //   AF(n) = the counter's current value, n
3    // Rep invariant:
4    //   n >= 0
5    // Safety from rep exposure:
6    //   n is an unexported int (immutable); no method returns a reference
to it.
7    func (c *Counter) checkRep() {
8      if c.n < 0 {
9        panic("demo: Counter.n is negative")
10     }
11   }
```

Listing 2: 三段注释 + `checkRep` 的通用形状：AF 可代入、RI 是 rep 上的谓词、安全性逐字段；示例与 `KV` 无关

1.5. 四、Go 手段（三）：表达“存在性”

Go 给了五种相关手段，它们回答的问题不同：

手段	写法	能回答什么	不能回答什么
单返回值	`v := m[k]`	键对应的值；缺失时是零值	键到底在不在
comma-ok	`v, ok := m[k]`	值 + 是否存在	—
只查存在	`_, ok := m[k]`	是否存在	值是什么
删除	`delete(m, k)`	执行删除；nil map 或不存在的键都是 no-op	删没删到（没有返回值）
长度差	`len(m)` 前后对比	集合大小变没变	哪个键变了；对 nil map 也不适用

⚠️ **陷阱预警** 这张表是**工具箱**，不是“某道练习该选哪个”的答案。选哪个取决于你的规格：`deleted` 要表达什么、未命中算不算错误、调用方需要区分什么。这些是规格决策，必须由你拍板并用测试固定。

1.5.1. 语言事实（可查证）

1. `delete` 的签名与 no-op 语义：`go doc builtin.delete`；
2. comma-ok 是语言规范里的索引表达式两种形式之一：`/usr/lib/go/doc/go_spec.html` 的 Index expressions 一节；
3. nil map 写入 panic：`/usr/lib/go/src/runtime/map.go:21` 的错误文本，写入路径在 `mapassign`。

1.6. 五、Go 手段（四）：表达“所有权”

切片不是数组，而是一个**描述符**：指针、长度、容量。赋值只复制描述符，不复制底层数组。于是同一个底层数组可以有两个“名字”，一端修改会影响另一端——这叫**别名（aliasing）**。

三种所有权策略，各有代价：

策略	做法	优点	代价
借用 + 约定	保存/返回原切片，文档写明“调用方不得修改”	零拷贝、快	违约时内部状态静默改变
防御性拷贝	进出都用 `bytes.Clone` / `slices.Clone`	边界真正封死	每次操作多一次分配与复制
不可变表示	内部存 `string` 或不可变结构	安全性论证最简单	存取时要做转换，可能再复制一次

`go doc` 的原文要点：

1. `bytes.Clone`：返回 `b[:len(b)]` 的副本；`Clone(nil)` **返回 nil**；
2. `slices.Clone`：浅拷贝，**保留 nil 性**；
3. `maps.Clone`：浅拷贝，键值用普通赋值。

```
type Buffer struct{ data []byte }

// 借用：调用方必须保证 p 在 Buffer 生命周期内不被修改
func (b *Buffer) SetBorrowed(p []byte) { b.data = p }

// 拷贝：与调用方彻底断开
func (b *Buffer) SetCopied(p []byte) { b.data = bytes.Clone(p) }

func (b *Buffer) GetBorrowed() []byte { return b.data }
func (b *Buffer) GetCopied() []byte   { return bytes.Clone(b.data) }
```

Listing 3: 四种组合的教学示例：借用/拷贝 × 进/出；它展示的是策略与代价，不是对项目某个方法的建议

1.6.1. 一个容易忽略的对称性

`string([]byte)` 会复制字节（规范原文在 `/usr/lib/go/doc/go_spec.html:5630`），所以把 `[]byte` 转成 `string` 当 map 键时，键与调用方是断开的；但把 `[]byte` 直接存进 map 时，值并没有复制。**同一个函数里，键安全、值不一定安全**——安全性论证必须逐字段说清，不能笼统写“没有暴露”。

1.7. 六、Go 手段（五）：表达“错误”与“不存在”

`error` 是一个接口，**nil 表示“没有错误”**（`go doc builtin.error` 原文）。它不是“没有结果”的通道。

两种契约形状：

形状	签名	表达什么	调用方怎么用
存在性	`(T, bool)`	“查到了没有”是正常结果	第二个返回值分支

错误	`(T, error)`	“操作失败了”是异常结果	`err != nil` 分支
两者都要	`(T, bool, error)`	正常但空 + 真正的失败	两个分支都要处理

如果让 `error` 同时表示“不存在”和“操作失败”，调用方就失去了区分能力：`err != nil` 无法说明是“没有这个键”还是“磁盘坏了”。补救办法是哨兵错误（sentinel error）加 `errors.Is`，但这会把一个存在性问题升级成错误分类问题。

相关 API（可查证）：

1. `go doc errors.Is`：沿 `Unwrap` 链匹配目标错误；
2. `go doc errors.As`：沿链找第一个可赋值给目标的错误；
3. `go doc builtin.panic`：panic 用于不可恢复状态，不是可预期失败。

1.8. 七、Go 手段（六）：把测试分区写出来

分区的三条性质（不交、完备、非空）与边界值单列，是**设计**；Go 只是把它表达出来。常用形状是表驱动 + 命名子测试：

```
1    func TestCounterAdd(t *testing.T) {
2      tests := []struct {
3        name         string
4        start, delta int
5        want         int
6      }{
7        {"zero", 0, 0, 0},
8        {"positive", 0, 3, 3},
9        {"from existing", 2, 3, 5},
10     }
11
12     for _, tt := range tests {
13       t.Run(tt.name, func(t *testing.T) {
14         c := Counter{n: tt.start}
15         c.Add(tt.delta)
16         if got := c.Value(); got != tt.want {
17           t.Fatalf("Counter{%d}.Add(%d) = %d, want %d", tt.start, tt.delta,
got, tt.want)
18         }
19       })
20     }
21   }
```

Listing 4: 表驱动 + `t.Run` + 失败消息四要素（函数、输入、got、want）；`go doc testing.T.Run` 说明子测试在独立 goroutine 中运行

两个配套事实：

1. **白盒与黑盒**：同包测试（`package db`）能访问非导出标识符，是白盒；测试包（`package db_test`）只能走公开 API，是黑盒。断言应按公开契约写，白盒能力留给检查内部不变量。

2. `t.Helper()`：标记辅助函数，失败时跳过辅助函数的栈帧，报错位置指向调用点（`go doc testing.T.Helper`）。

1.9. 八、当前步骤的“语言工具箱”

下面只列**可用手段**与**你必须自己回答的问题**，不列选择、不列步骤。

当前步骤	可用手段	各手段的语义差异	你必须自己回答的问题
练习 0（文档）	字段旁注释、`checkRep`、`panic`	注释可代入 vs 口号；RI 是谓词 vs 抽象描述	抽象值是什么？哪些 rep 状态合法？客户端能拿到哪些可变部分？
练习 1（删除）	comma-ok、`delete`、`len` 差	前两者能回答存在性；`delete` 无返回值	`deleted` 的语义是什么？删不存在的键算不算失败？
练习 2（未命中）	单返回值、comma-ok、`error`	零值无法区分“缺失”与“存的零值”；error 是错误通道	未命中是正常结果还是错误？调用方需要区分什么？
测试分区	表驱动、`t.Run`、testify	子测试隔离；断言强度不同	输入空间有哪些维度？合法输入怎么定义？边界在哪？
安全性论证	借用、`bytes.Clone`、不可变 rep	零拷贝 vs 分配；约定 vs 强制	切片所有权归谁？违约时谁负责？

> 📌 **核心要点** 这张表最容易被误读成“照着手段清单选一个”。实际上每一行都有一列**你必须自己回答的问题**——那一列才是练习的核心。手段只是表达方式；规格没定，选什么手段都是在猜。

1.10. 九、常见错误（概念类 / Go 类 / 学习类）

1. **概念类**：AF 写成口号；RI 引用抽象值；把“测试通过”当“规格正确”；把“未命中”当错误。
2. **Go 类**：nil map 写入 panic；切片别名；`string` 转换复制而值不复制；同包白盒测试直接改内部；`panic` 用于可预期失败；`error` 字符串大写或带标点。
3. **学习类**：把技能里的示例直接搬进项目；把“我懂了”当“我写出来了”；跳过文档直接写实现；用“能编译”代替“规格一致”。

1.11. 十、关键点与自检

一句话：**AF 让内部数据“可读”，RI 让非法状态“进不来”，安全性论证让客户端“伸不进手”；Go 用注释、非导出字段、`checkRep`、comma-ok、`Clone` 与 `error` 分别表达这四件事。**

自检四问：

1. 我的 AF 能代入一个具体 rep 值并算出唯一抽象值吗？
2. 我的 RI 每条都是 rep 上的布尔谓词吗？有没有偷偷引用抽象值？
3. 我的安全性论证覆盖了每个字段、每个参数与返回值吗？
4. 我的测试分区覆盖了空、零、重复、未初始化这些边界吗？

1.12. 十一、概念变式（不指向实现）

1. 如果 AF 对一个合法 rep 值没有定义，问题出在 AF 还是 RI？两者的一致性条件是什么？
2. 同一个 rep 类型，能不能配两组不同的 RI/AF？如果能，说明“类型决定一切”这句话错在哪？
3. 一个不可变的 rep 为什么让安全性论证变简单？如果 rep 里有一个可变字段，论证多了哪一步？
4. `(T, bool)` 与 `(T, error)` 两种契约，分别把“不存在”归到哪一类？调用方的代码形状有什么不同？

1.13. 十二、同类归类

概念	课程位置	Go 表达	对应技能
ADT 形状	6.102 R06	结构体 + 非导出字段 + 方法	`golang-structs-interfaces`
AF/RI/表示暴露	6.102 R07	字段旁注释 + `checkRep` + 防御性拷贝	`golang-safety`、`golang-naming`
测试分区	6.102 R02	表驱动 + 子测试 + 边界	`golang-testing`
规格与错误	6.102 R04/R05	bool/ok 与 error 的分工	`golang-error-handling`
工具链	6.102 R07 recipes	`go doc` / `go vet` / `go test` / `gofmt`	`golang-troubleshooting`

1.14. 十三、Go 机制补充：查证清单

这一节的事实都有离线出处，别凭记忆：

1. `go doc builtin.delete`、`go doc builtin.error`、`go doc builtin.panic`；
2. `go doc bytes.Clone`、`go doc slices.Clone`、`go doc maps.Clone`；
3. `go doc errors.Is`、`go doc errors.As`；
4. `go doc testing.T.Run`、`go doc testing.T.Helper`；
5. 语言规范：`/usr/lib/go/doc/go_spec.html`（索引表达式、切片转字符串在 5630 行附近）；
6. 运行时：`/usr/lib/go/src/runtime/map.go:21`（nil map 写入的错误文本）、`mapassign`（同文件第 124 行）。

三层理解模板（讲每个机制时都按这个顺序）：

1. **为什么存在**：它解决什么痛点（例如 nil map 让零值可用，但写入需要初始化）；

2. **语言/运行时怎么实现**：编译期规则还是运行时检查（例如非导出字段是编译期，nil map panic 是运行时）；
3. **日常怎么写才地道**：在创建者里初始化、在 API 边界决定所有权、正常结果不走 error。

1.15. 十四、本次操作

1. **重讲内容**：ADT 三个问题、AF/RI/表示暴露的关系、Go 的 ADT 形状、注释与 `checkRep` 的位置、存在性五手段、所有权三策略、错误两契约、表驱动测试、当前步骤工具箱。
2. **边界**：没有贴 `KV` 的任何实现、没有指定练习 0/1/2 的选择、没有给下一步；示例用 `Counter`、`Buffer`、`Counter` 测试等通用类型。
3. **可查证**：`go doc` 的六条输出、规范行号、`runtime/map.go` 行号都已给出。
4. **下一步**：由你决定——按工具箱自己选、写完后交作业，或继续问概念。若要直接看实现，明确说解除契约。
5. **文件精简**：按用户要求，本文件只保留本节；此前的其他主题已移除，完整备份在 `/tmp/2026-10-24.full.typ`。

2. map 类表示的抽象函数怎么写（AF for Map-shaped Reps）

2.1. 费曼直觉

AF 是翻译官：它把内部那堆数据翻译成客户端听得懂的话。map 只是“键 → 值”的容器，所以写 map 的 AF，本质上就是回答一句话：

> 📌 **核心要点 “对每一个键，客户端看到的是什么？”** 把所有键的答案合起来，就是抽象值。

如果抽象值本身就是“一张有限键值表”，那么 AF 几乎是把 map 原样翻译过去；如果抽象值是别的结构（集合、计数器、稀疏数组），AF 就要说明“这些键值对怎么读成那个结构”。

2.2. 硬核机制

2.2.1. 一、AF 的左边与右边

写 AF 之前先固定两件事：

1. **左边**是 rep 里的字段，例如 `mem`；如果有多个字段，就按字段名并列写；
2. **右边**必须是抽象值空间里的一个值，不能是 rep 的另一种说法。

判定标准只有一条：**把具体的 rep 值代进去，右边能算出唯一一个抽象值**。算不出来，或者代进去右边不变，就不是 AF。

2.2.2. 二、map 类 AF 的三种标准写法

设 rep 是 `m : map[K]V`。三种写法在数学上等价，选哪一种取决于你把抽象值空间定义成什么。

写法 A：映射构造式（map comprehension）

```
AF(m) = { k -> m[k] | k in dom(m) }
```

Listing 5: 读作：对 `m` 的每一个键 `k`，抽象值把 `k` 映射到 `m[k]`；`dom(m)` 是 `m` 的键集合

适合抽象值 = **有限映射**（有限函数）。这是最常用的一种。

写法 B：函数式（lambda）

```
AF(m) = (lambda k. m[k])  with domain dom(m)
```

Listing 6: 把抽象值看成一个部分函数：定义域是 `dom(m)`，在定义域上取值为 `m[k]`

适合抽象值本身就是“部分函数”的场合；与写法 A 只是记号差异。

写法 C：条目集合式

```
AF(m) = { (k, m[k]) | k in dom(m) }
```

Listing 7: 把抽象值看成“键值对的无序集合”；适合抽象值不强调“映射”而强调“条目”的场合

三种写法都满足同一个约束：**同样的键值对，无论 map 的容量、哈希种子、遍历顺序如何，AF 的结果必须相同。**

2.2.3. 三、空 map、nil map 与定义域

AF 只对**合法**的 rep 值有定义；哪些 rep 值合法由 RI 决定。于是：

1. 空 map（`map[K]V{}`）通常映射到**空映射**或**空集合**——这是有定义的；
2. nil map 是否在 AF 的定义域内，取决于 RI 是否允许它；如果 RI 不允许，AF 不必讨论 nil；
3. `AF` 不需要描述“空 map 和 nil map 的区别”；如果两者都合法，AF 应给出同样的抽象值（空），区别属于 RI 的范畴。

> ⚠ **陷阱预警** 不要把 rep 的细节写进 AF：容量（capacity）、哈希桶、遍历顺序、nil 与否，都不是抽象值的一部分。AF 只回答“客户端看到什么”。

2.2.4. 四、当抽象值不是“映射”时

map 的 AF 不一定长得像“一张表”。三个通用例子：

1. **集合**：`map[string]bool` 表示“名字集合”时，可以只取值为 true 的键：`AF(m) = { k | k in dom(m) and m[k] = true }`；
2. **计数器**：`map[string]int` 表示“单词 → 出现次数”时，AF 仍是映射构造式，但抽象值是“计数表”；
3. **稀疏数组**：`map[int]string` 表示“下标 → 值的部分函数”时，AF 的函数式写法最自然。

如果 rep 有多个字段（例如 map 加一个长度字段），而某些字段不参与抽象值，AF 里可以忽略它们；但要在注释里说明“哪个字段被忽略、为什么”。这属于设计决策，不是语法问题。

2.2.5. 五、常见错误

1. **口号式**：`AF(m) = 一张表`——代入任何 `m`，右边都不变；
2. **循环式**：`AF(m) = m 里的键值对`——用 rep 解释 rep，没有翻译；
3. **把所有权写进 AF**：值是否共享、是否拷贝，属于安全性论证，不属于 AF；
4. **把 nil 写进 AF**：nil 是否合法是 RI 的事；
5. **忽略空 map**：空 map 也是一个合法 rep 值，AF 必须给出结果。

2.2.6. 六、自检三问

1. 代入一个具体的 map（比如 `{"a": 1, "b": 2}`），AF 能算出唯一结果吗？
2. 同一个抽象值的两种不同 rep（容量不同、插入顺序不同），AF 结果相同吗？
3. 非法 rep（nil、值类型不合法）是否被 RI 排除，而不是靠 AF 兜底？

2.3. 当前步骤：`mem` 的三个待答问题

对 `KV.mem`，AF 的**结构**可以套上面的写法 A/B/C，但**内容**必须由你回答：

1. **抽象值空间是什么？** 是“有限键值映射”，还是别的结构？
2. **键和值分别怎么读？** 值是字节串本身，还是解码后的东西？
3. **空 map、nil map 在不在定义域？** 这由 RI 决定，不是 AF 决定。

给一个**结构模板**（不是答案）：

```
AF(mem) = { <key> -> <value> | <condition> }
```

Listing 8: 把三个尖括号填成你自己的设计；填完之后用上一节的三问自检

> 📌 **核心要点** 到这里为止，能给的只有“怎么写”与“怎么验”；“写什么”是你的规格决策。三个问题答完，AF 自然就出来了。

2.4. 关键点

一句话：map 的 AF 就是“逐键翻译”——对每个键说明客户端看到什么，再说明空 map 落到哪个抽象值；剩下的细节（nil、容量、所有权）分别归 RI 与安全性论证。

2.5. 变式检验

1. 如果抽象值是“集合”，而 rep 是 `map[string]bool`，AF 的右边应该怎么写？值为 false 的键怎么办？
2. 如果 rep 有两个字段（map + 长度），长度字段不参与抽象值，AF 里要不要提它？为什么？
3. 空 map 与 nil map 都合法时，AF 的结果应该相同还是不同？这与 RI 的哪条子句有关？
4. 如果 AF 写成 `AF(m) = m`，它在什么情况下是合法的？什么情况下是循环定义？

2.6. 同类归类

REP 形状	AF 的典型写法	注意
map	逐键映射 / 部分函数 / 条目集合	空 map、nil、遍历顺序无关
slice/数组	下标 → 元素；或前缀/区间解释	长度、容量、nil 切片
结构体多字段	按字段组合成抽象值，可忽略不参与字段	忽略字段要说明理由
字符串	字符集合、区间、编码后的值	重复字符、顺序是否重要

2.7. Go 语言机制补充

1. map 是引用类型，零值是 nil；读 nil map 返回零值，写入 panic（`/usr/lib/go/src/runtime/map.go:21`）；

2. map 的迭代顺序是随机的（规范不保证），所以 AF 绝不能依赖遍历顺序；
3. `len(m)` 是键值对数量；`delete` 对 nil map 与缺失键都是 no-op（`go doc builtin.delete`）；
4. `maps.Clone` 是浅拷贝（`go doc maps.Clone`）；键值是否共享，属于安全性论证。

2.8. 本次操作与当前状态

1. **新增主题**：map 类 rep 的 AF 三种写法、空 map/nil map 与定义域、非映射抽象值的例子、常见错误、自检三问，以及对 `mem` 只给结构模板、不给内容。
2. **未改动**：`internal/db/kv.go`、`internal/db/kv_test.go`；没有填 `KV` 的 AF 内容，没有给练习答案。
3. **验证**：`typst compile` 零错误；`go vet ./...` 通过。
4. **下一步**：你把三个待答问题写出来，或者直接写进代码后交作业，我按契约给诊断。

Index of Listings

Listing 1 Go 里 ADT 的最小形状：非导出字段 + 方法；没有类，封装靠包边界与字段大小写 4
Listing 2 三段注释 + `checkRep` 的通用形状：AF 可代入、RI 是 rep 上的谓词、安全性逐字段；示例与 `KV` 无关 5
Listing 3 四种组合的教学示例：借用/拷贝 × 进/出；它展示的是策略与代价，不是对项目某个方法的建议 6
Listing 4 表驱动 + `t.Run` + 失败消息四要素（函数、输入、got、want）；`go doc testing.T.Run` 说明子测试在独立 goroutine 中运行 7
Listing 5 读作：对 `m` 的每一个键 `k`，抽象值把 `k` 映射到 `m[k]`； `dom(m)` 是 `m` 的键集合 11
Listing 6 把抽象值看成一个部分函数：定义域是 `dom(m)`，在定义域上取值为 `m[k]` . 11
Listing 7 把抽象值看成“键值对的无序集合”；适合抽象值不强调“映射”而强调“条目”的场合 12
Listing 8 把三个尖括号填成你自己的设计；填完之后用上一节的三问自检 13